

MODULE: VECTORIZATION (NUMPY)

Optimization of tensor operations through the elimination of iterative loops

1 The Essence of Vectorization

In classical programming (e.g., in standard Python), operations on lists are performed element by element, using a `for` loop. For interpreted languages like Python, each loop iteration involves a massive time penalty (so-called *overhead*): the interpreter must check the variable type, allocate memory, and call the appropriate function every single time.

Vectorization is a technique of delegating operations on entire data structures (vectors, matrices) directly to low-level libraries written in C or Fortran (like BLAS or LAPACK), which under the hood utilize SIMD (*Single Instruction, Multiple Data*) processor instructions. Thanks to this, the processor performs a single mathematical operation on multiple memory blocks simultaneously.

2 Mathematical Perspective

Instead of treating the problem as an algorithmic iteration summing the squares of n scalars:

$$S = \sum_{i=1}^n x_i^2$$

we switch to the language of linear algebra. We define a column vector $\mathbf{x} \in \mathbb{R}^n$:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

The sum of squares of the elements of this vector is simply the dot product of the vector $\mathbf{x}$ with itself, which we write as:

$$S = \mathbf{x}^T \mathbf{x}$$

Vectorization allows our code to work exactly this way – it applies operations immediately on the entire object in the $\mathbb{R}^n$ space.

3 Knowledge Check: Sum of squares of a million numbers

Your control task from the roadmap is to calculate the sum of squares of a million numbers using vectorization. The script below implements this solution in two ways: classically (loop) and via vectorization (NumPy), demonstrating the difference in execution time.

```
1      import numpy as np
2      import time
3
4      # Generate a vector of a million random numbers from the interval [0,
5          1)
6      N = 1_000_000
7      data_list = [np.random.random() for _ in range(N)]
8      data_vector = np.array(data_list)
9
10     print(f"Data: vector of {N} elements in R^n space\n")
```

```

11      # --- METHOD 1: TRADITIONAL FOR LOOP (PYTHON DEVELOPER'S NIGHTMARE)
12      ---
13      start_time = time.perf_counter()
14      sum_loop = 0
15      for x in data_list:
16          sum_loop += x**2
17      time_loop = time.perf_counter() - start_time
18
19      print(f"FOR Loop:      Sum = {sum_loop:.2f}")
20      print(f"Time (loop):    {time_loop:.5f} seconds\n")
21
22      # --- METHOD 2: NUMPY VECTORIZATION (DEEP TECH STANDARD) ---
23      # Checkpoint solution: we operate on the object as a whole
24      start_time = time.perf_counter()
25
26      # x**2 works vectorized (squares each element in C)
27      # np.sum() instantly sums the memory area
28      sum_vector = np.sum(data_vector**2)
29
30      # Alternatively, using a pure dot product (even faster):
31      # sum_vector = np.dot(data_vector, data_vector)
32
33      time_vector = time.perf_counter() - start_time
34
35      print(f"NumPy vector:  Sum = {sum_vector:.2f}")
36      print(f"Time (NumPy):  {time_vector:.5f} seconds")
37      print(f"Speedup: {time_loop / time_vector:.1f}x faster!")

```

Listing 1: Performance comparison: FOR Loop vs. NumPy Vectorization

4 R&D Conclusions

Switching from loops to matrix operations is not just a matter of syntactical convenience. The output of the above script usually shows a **50x to even 150x faster execution** of the vectorized variant.

In the context of upcoming FEM simulations (where we will operate on stiffness matrices of sizes around $10^5 \times 10^5$) or training PINN networks, using a for loop for error propagation would bring computations to a complete halt. Vectorization is the foundation upon which the entirety of SciML is built.

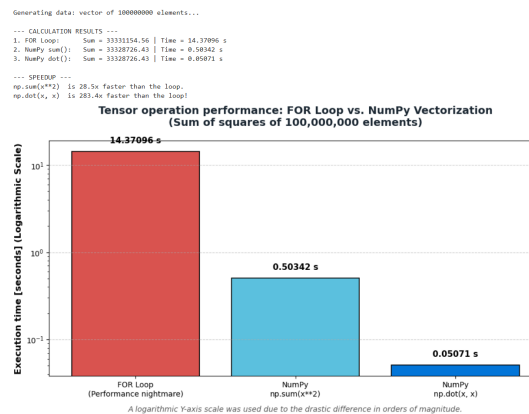


Figure 1: The logarithmic scale reveals the performance gulf between the FOR loop and hardware vectorization in NumPy.